Übungsblatt 1: Zeitverträge, Polling und eine Tasteranzeige

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Aufgabe 1: Einen Zeitvertrag für eine Laborsteuerung formulieren (8 Punkte)

Nach einem erfassten Trigger muss eine Laborsteuerung binnen $D = 80\,\mu s$ den richtigen Startimpuls am Softwareausgang ausgeben. Jede Fristverletzung gilt als Fehler. Ein unabhängig berechneter Positionswert ist nach Ende seines 4-ms-Messfensters wertlos; höchstens ein fehlender Wert je 200 Fenster ist erlaubt. Eine verspätete Raumtemperaturanzeige bleibt nützlich.

1. Benennen Sie Anfang, Abschluss und Zeitbedingung des Startimpulses. Warum reicht „möglichst schnell“ nicht? (2 Punkte)
2. Ordnen Sie Startimpuls, Positionswert und Temperaturanzeige Hard, Firm und Soft zu. Begründen Sie anhand des jeweiligen Vertrags. (3 Punkte)
3. Der Trigger wird bei $r = 25\,\mu s$ erfasst. Bestimmen Sie die absolute Deadline und bewerten Sie richtige Startimpulse bei 105 und 107 µs. (2 Punkte)
4. Ein Impuls liegt schon bei 50 µs an, hat aber den falschen Pegel. Ist der Gesamtvertrag erfüllt? (1 Punkt)

Musterlösung: Anfang ist die festgelegte Triggererfassung, Abschluss der richtige Startimpuls am vereinbarten Softwareausgang. Zeitbedingung $f - r \leq 80\,\mu s$. „Schnell“ definiert weder Endpunkte noch eine Grenze. Startimpuls Hard, Positionswert Firm, Temperaturanzeige Soft: Ausschlaggebend sind erlaubte Fristverletzungen und Restnutzen, nicht die Gerätebezeichnung.

$d = r + D = 25 + 80 = 105\,\mu s$. Bei 105 ist die inklusive Frist eingehalten, bei 107 um 2 µs verletzt. Der falsche Pegel bei 50 erfüllt nur die Zeitbedingung; funktionale und zeitliche Korrektheit müssen gemeinsam gelten.

Aufgabe 2: Antwortzeit, eigene Arbeit und Jitter trennen (8 Punkte)

Eine Bearbeitung wird bei 20 µs freigegeben, beginnt bei 35 µs und rechnet bis 65 µs. Von 65 bis 90 µs wird sie durch einen Interrupt unterbrochen. Danach rechnet sie bis zum Abschluss bei 110 µs. Absolute Deadline: 125 µs. Alle zusätzlich nötigen Verwaltungskosten seien bereits in diesen Zeitpunkten enthalten.

Eine andere, periodische Ausgabe hat folgende Zeitpunkte:

Ereignisnummer k	0	1	2	3	4
Soll in ms	0	6	12	18	24
Ist in ms	0	6,4	12,1	18,3	24,2

1. Zeichnen Sie die Bearbeitung auf einer Zeitachse. Berechnen Sie eigene CPU-Zeit, Startverzug, Antwortzeit und Reserve bis zur Deadline. (4 Punkte)
2. Berechnen Sie die vier Istperioden, ihren Mittelwert und die Periodenspanne als Maximum minus Minimum. (2 Punkte)
3. Berechnen Sie die Abweichung jedes Istzeitpunkts von seinem Sollzeitpunkt und deren Spanne. Warum ist das eine andere Größe? (1 Punkt)
4. Beweist das größte Ergebnis einer endlichen Messreihe die WCET, die größte mögliche eigene Ausführungszeit unter festgelegten Bedingungen? (1 Punkt)

Musterlösung: Eigene Rechenabschnitte $[35, 65)$ und $[90, 110)$ µs; eigene CPU-Zeit $30 + 20 = 50\,\mu s$. Startverzug $35 - 20 = 15\,\mu s$, Antwortzeit $110 - 20 = 90\,\mu s$, Reserve $125 - 110 = 15\,\mu s$. Kontrolle: Anfangswarten 15 plus Interrupt 25 plus eigene Arbeit 50 ergeben 90 µs. Die relative Deadline beträgt 105 µs, ist also eingehalten.

Istperioden: 6,4; 5,7; 6,2; 5,9 ms. Mittelwert $24{,}2/4 = 6{,}05\,ms$, Spanne $6{,}4 - 5{,}7 = 0{,}7\,ms$. Phasenabweichungen: 0; 0,4; 0,1; 0,3; 0,2 ms, Spanne 0,4 ms. Phase vergleicht Ist und Soll desselben Ereignisses, Periode benachbarte Istereignisse. WCET bedeutet Worst-Case Execution Time. Ein Messmaximum deckt nicht automatisch alle zulässigen Eingaben, Pfade und Hardwarezustände ab.

Aufgabe 3: Eine Hauptschleife mit Polling beurteilen (8 Punkte)

Die Hauptschleife prüft einen gespeicherten Auftrag, reagiert gegebenenfalls und bearbeitet anschließend einen Datenblock. Obergrenzen: Prüfung 0,15 ms, Reaktion 0,25 ms, Datenblock 1,60 ms. Ein Kern, keine Interrupts, höchstens ein relevanter Auftrag; die Erkennung liegt am Ende der Prüfung. Der Auftrag bleibt bis dahin gespeichert. Relative Frist: 3 ms. Für die konservative Antwortgrenze verwenden Sie eine ganze maximale Runde bis zur Erkennung und addieren anschließend die Reaktion.

1. Berechnen Sie maximale Rundenzeit und Antwortgrenze. Ist die Frist im Modell nachweisbar? (2 Punkte)
2. Der Datenblock benötigt nun 4,60 ms. Rechnen Sie erneut und beschreiben Sie eine konkrete Ankunftsphase mit Fristverletzung. (2 Punkte)
3. Ohne Speicherung wird nur der momentane Eingangspegel gelesen: Abfragen bei 0, 2 und 4 ms, HIGH nur von 0,7 bis 1,1 ms. Wird das Ereignis verspätet erkannt oder vollständig verloren? Was verändert ein Ereignislatch? (2 Punkte)
4. Zerlegen Sie die Datenarbeit in Schritte von höchstens 0,40 ms mit erneuter Eingangsprüfung nach jedem Schritt. Bestimmen Sie die neue Grenze. Welche Information muss die Datenbearbeitung zwischen den Runden speichern? (2 Punkte)

Musterlösung: Runde $0{,}15 + 0{,}25 + 1{,}60 = 2{,}00\,ms$, konservativ $R \leq 2{,}25\,ms < 3\,ms$. Mit langem Block: Runde 5,00 ms, Antwortgrenze 5,25 ms. Die große Grenze allein beweist noch keinen tatsächlichen Miss. Ein Auftrag unmittelbar nach seiner Prüfung vor Beginn der 4,60-ms-Datenarbeit wartet aber bereits fast 4,60 ms und verletzt 3 ms tatsächlich.

Alle Pegelabfragen sehen LOW: Das Ereignis geht verloren und wird nicht später bearbeitet. Ein Latch hält einen Hinweis bis zur Abfrage fest, zählt aber nicht beliebig viele Ereignisse. Mit kurzen Schritten: Runde 0,80 ms, Antwortgrenze 1,05 ms. Die Datenbearbeitung speichert etwa den nächsten Index und ihr Zwischenergebnis. Ein kleiner Schritt beendet nicht automatisch die gesamte Datenarbeit.

Aufgabe 4: Timer, Interruptpriorität und Erfassung unterscheiden (8 Punkte)

1. Ein Timer hat den bestätigten Timer-Kerntakt 96 MHz, $PSC = 9599$ und $ARR = 1999$. Berechnen Sie mit $f_{\mathrm{UPD}} = f_{\mathrm{TIM}}/((PSC + 1)(ARR + 1))$ Updateabstand und vollständige Blinkfrequenz bei einem Toggle pro Update. (3 Punkte)
2. Ein Mess-IRQ mit Präemptionsrang 2 und Subpriorität 3 läuft bereits. Ein Taster-IRQ mit Rang 2 und Subpriorität 0 wird angefordert. Kann er allein wegen der Subpriorität unterbrechen? Was ändert ein zulässiger Präemptionsrang 1? Kleinere Zahlen sind dringlicher; eine passende Gruppierung ist vorausgesetzt. (2 Punkte)
3. Ein Hardware-Pendingbit merkt mindestens ein Ereignis. Drei gleichartige Ereignisse treffen vor seiner Bedienung ein; das Bit bleibt gesetzt. Der anschließende Callback erhöht einen `volatile`-Zähler einmal. Sind damit drei physische Ereignisse gezählt? Begründen Sie. (2 Punkte)
4. Eine Anforderung trifft bei noch 18 µs Maskierung ein. Danach benötigen Eintritt und Reaktion disjunkt weitere 4 und 6 µs. Prüfen Sie eine Gesamtfrist von 25 µs. (1 Punkt)

Musterlösung: $f_{\mathrm{UPD}} = 96 \cdot 10^6/(9600 \cdot 2000) = 5\,\mathrm{Hz}$. Updateabstand 200 ms, zwei Toggles pro An-Aus-Zyklus ergeben 2,5 Hz Blinken. CPU-Takt und tatsächlicher Timer-Kerntakt dürfen nicht ohne Prüfung gleichgesetzt werden.

Subpriorität allein unterbricht keinen laufenden gleichrangigen Handler; sie ordnet gleichzeitig anstehende Anforderungen. Ein dringlicherer zulässiger Rang 1 kann Rang 2 unter den Maskierungsbedingungen verdrängen. Ein einzelnes Pendingbit kann mehrere Ereignisse vor dem Callback zusammenfassen. Ein einmaliger Zählerzugriff erfasst nur den bedienten Hinweis; `volatile` rekonstruiert keine zuvor verlorene Anzahl. Gesamtantwort $18 + 4 + 6 = 28\,\mu\mathrm{s}$, Fristverletzung um 3 µs.

Aufgabe 5: Einen unvollständigen Prüfbericht bewerten

Ein Test protokolliert mit einer gemeinsamen Modelluhr Freigaben bei 0, 20, 40 und 60 ms. Zugehörige richtige Ausgaben sind bei 6, 27 und 49 ms dokumentiert; für den vierten Auftrag fehlt der Abschluss. Erlaubt sind höchstens 8 ms Antwortzeit.

1. Bestimmen Sie die drei auswertbaren Antwortzeiten und ihren Mittelwert. Gilt diese Zahl automatisch als Mittelwert aller vier Aufträge?
2. Welche Fristverletzung ist bereits belegt? Wie bewerten Sie den vierten Auftrag ohne erfundenen Nullwert?
3. Eine LED blinkte während des Tests gleichmäßig. Ändert das Ihre Bewertung?

Musterlösung: Auswertbare Antworten: 6,7,9 ms, Mittelwert $22/3 \approx 7{,}33\,\mathrm{ms}$ nur für diese drei. Für alle vier ist ohne weitere Daten kein Mittelwert bestimmbar. Der dritte Auftrag verletzt die Frist um 1 ms und widerlegt die behauptete Garantie unter diesen Testbedingungen. Beim vierten ist ohne Abschluss beziehungsweise weitere Beobachtung die vollständige Antwort nicht bestimmbar; er darf nicht als rechtzeitig oder als Nullzeit verbucht werden. Gleichmäßiges Blinken betrifft eine andere Beobachtung und repariert keinen belegten Miss.

Aufgabe 6: Eine Tasteranzeige mit Zeitablauf selbst implementieren

Programmieren, etwa 30–40 Minuten. CM4-Basis: `lab01_button.c/.h` und `button_step.c`. **B1/PC13** liefert entprellte Zustände: Startzustand sofort, danach 20 ms Stabilitätszeit. **LD1/PB0** ist Ihre Anzeige; **LD2/PE1** zeigt den Fortschritt derselben Hauptschleife. Kein RTOS.

Vorhandene Schnittstelle	Bedeutung
`Lab01_ButtonPressed()`	null losgelassen, sonst gedrückt
`HAL_GetTick()`	HAL-Zeit als `uint32_t`, hier 1 Tickwert = 1 ms
`Lab01_ButtonLed(on)`	LD1 setzen: null aus, sonst ein
`LAB01_LIGHT_TICKS`	Anzeigedauer: 800 Tickwerte
`lab01_presses`	Betätigungszähler, Ihr Schritt ist einziger Schreiber

1. Planen Sie `Lab01_ButtonStep()` in `button_step.c`: Neue Betätigung bedeutet 800 Tickwerte Licht und ein Zählschritt. Erneutes Drücken startet die Zeit neu; Halten verlängert sie nicht. Ein beim Start gedrückter Taster zählt einmal.
2. Implementieren Sie Zustandswechselerkennung und Zeitablauf. Welche Variablen müssen zwischen zwei Aufrufen erhalten bleiben? Nutzen Sie vorzeichenlose Zeitdifferenzen. Bei abgelaufenen 800 Tickwerten soll die Anzeige ausgeschaltet werden.
3. Kehren Sie nach begrenzter Arbeit zurück: kein `HAL_Delay`, keine Warteschleife. Prüfen Sie kurzes Drücken, Halten über 800 Tickwerte und erneutes Drücken während der Leuchtphase. LD2 muss weiterlaufen.
4. Ergänzen Sie die Tabelle vor dem Boardtest: bereits entprellte, idealisierte Testeingaben; Werte jeweils nach dem Aufruf, vor Tick null alles aus/null. Die Zahlen sind keine gemessenen Boardzeiten.

Tickwert	Tasterzustand	LD1 nach dem Schritt	Zähler
0	losgelassen		
100	gedrückt		
250	weiter gedrückt		
360	losgelassen		
620	erneut gedrückt		
700	losgelassen		
1419	losgelassen		
1420	losgelassen		

5. Prüfen Sie zusätzlich den Start mit gehaltenem B1 sowie den Überlauffall: Beginn `0xFFFFFF00`, jetzt `0x00000220`. Ist die Anzeigezeit abgelaufen? Wie unterscheiden sich das Zählen entprellter Betätigungen und das Zählen elektrischer Flanken?

Basis: Projektanleitung, STM32CubeMX, VS Code mit STM32 VS Code Extension, **NUCLEO-H755ZI-Q / STM32H755ZIT6**, USB/ST-LINK. Funktion und tatsächlichen Prüfstatus festhalten; Diagnose bei angehaltenem CM4.

Quellen: Buttazzo, *Hard Real-Time Computing Systems*; ST UM2408 und RM0399; Arm CMSIS-Core. Papierwerte sind Lehrannahmen.

Musterlösung zu Aufgabe 6

Benötigt werden vorheriger Tasterzustand, Aktivkennzeichen und Beginn der aktuellen Leuchtphase. Nur der vorgegebene Hauptprogrammkontext verändert diese Daten. Die Variablen bleiben beispielsweise mit `static` zwischen Aufrufen erhalten.

```
#include "lab01_button.h"

void Lab01_ButtonStep(void)
{
    static int previous, active;
    static uint32_t since;
    const int pressed = Lab01_ButtonPressed();
    const uint32_t now = HAL_GetTick();
    if (pressed && !previous) {
        since = now;
        active = 1;
        ++lab01_presses;
    }
    previous = pressed;
    if (active && (uint32_t)(now - since) >= LAB01_LIGHT_TICKS) {
        active = 0;
    }
    Lab01_ButtonLed(active);
}
```

Tickwert	LD1	Zähler	Begründung
0	aus	0	noch keine Betätigung
100	ein	1	erste Betätigung, Beginn bei 100
250	ein	1	Halten zählt nicht erneut
360	ein	1	Loslassen beendet die laufende Zeit nicht
620	ein	2	erneute Betätigung, neuer Beginn bei 620
700	ein	2	Zeit läuft weiter
1419	ein	2	seit letzter Betätigung 799 Ticks
1420	aus	2	genau 800 Ticks abgelaufen

Ist der Taster beim Start gedrückt, ist `previous` zunächst null; daher wird sofort eine Betätigung erkannt. Bleibt er gedrückt, ist bei Starttick plus 800 die LED aus; der Zähler bleibt eins. Die Überlaufdifferenz ist `(uint32_t)(0x00000220 - 0xFFFFFF00) = 800`, also ist die Zeit abgelaufen. Die Formel setzt Beobachtungsabstände ohne unbeobachtete vollständige 32-Bit-Umläufe voraus.

Die Entprellung erzeugt stabile Zustandswechsel, keinen verlustfreien Flankenzähler. Ein sehr kurzer Rohpuls kann absichtlich verschwinden. Tickphase und spätere Ausführung beeinflussen reale LED-Zeitpunkte: Nach Ablauf wird erst beim nächsten Schritt ausgeschaltet. Eine Funktionsprüfung beweist keine exakte physische Leuchtdauer oder harte Antwortgrenze.